Tagless Final

Tagless Final · Console greet with Option test interpreter · Build with Waqar

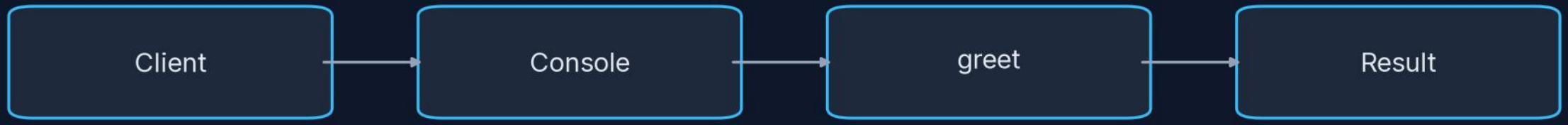
Today's Hook

- Problem: Console greet with Option test interpreter
- Pattern: Tagless Final — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- Console greet with Option test interpreter
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



Tagless Final: Console greet with Option test interpreter

Real-World Scenario

- Running example: Console greet with Option test interpreter
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```java
package com.buildwithwaqar.patterns.functional.tagless;

import com.buildwithwaqar.patterns.functional.tagless.TaglessFinal;
import com.buildwithwaqar.patterns.functional.typeclass.OptionKind;
/** Problem: greet via Console algebra; swap test vs prod interpreters. */
public final class UseCaseDemo {
  public static void main(String[] a) {
    System.out.println("=== Tagless Final problem: swappable Console ===");
    var console = new TaglessFinal.OptionConsole("Waqar");
    System.out.println("name=" + OptionKind.lower(TaglessFinal.greet(OptionKind.MONAD, console)).get());
  }
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```java
package com.buildwithwaqar.patterns.functional.tagless;

import com.buildwithwaqar.patterns.functional.typeclass.Kind;
import com.buildwithwaqar.patterns.functional.typeclass.Monad;
import com.buildwithwaqar.patterns.functional.typeclass.OptionKind;
import dev.zazr.control.Option;

/**
 * Tagless Final: programs are written against an algebra (interface of capabilities),
 * not a concrete effect type. Interpreters provide the algebra for Option, IO, etc.
 */
public final class TaglessFinal {
    private TaglessFinal() {}

    /** Unit type — Zazr Option rejects null, so Void is awkward; use a singleton. */
    public enum Unit { INSTANCE }

    public interface Console<F> {
        Kind<F, String> readLine();
        Kind<F, Unit> printLine(String s);
    }
```

Solution (2)

```
    public static <F> Kind<F, String> greet(Monad<F> M, Console<F> console) {
        return M.flatMap(console.printLine("What is your name?"), ignored ->
                M.flatMap(console.readLine(), name ->
                        M.flatMap(console.printLine("Hello, " + name), ignored2 ->
                                M.pure(name))));
    }

    /** Test interpreter using Option as a trivial effect (pre-canned input). */
    public static final class OptionConsole implements Console<OptionKind.Witness> {
        private final String input;
        public OptionConsole(String input) { this.input = input; }

        public Kind<OptionKind.Witness, String> readLine() {
            return OptionKind.lift(Option.some(input));
        }

        public Kind<OptionKind.Witness, Unit> printLine(String s) {
            return OptionKind.lift(Option.some(Unit.INSTANCE));
        }
    }
}
```

Runnable main

```java
package com.buildwithwaqar.patterns.functional.tagless;

import com.buildwithwaqar.patterns.functional.tagless.TaglessFinal;
import com.buildwithwaqar.patterns.functional.typeclass.OptionKind;
/** Problem: greet via Console algebra; swap test vs prod interpreters. */
public final class UseCaseDemo {
  public static void main(String[] a) {
    System.out.println("=== Tagless Final problem: swappable Console ===");
    var console = new TaglessFinal.OptionConsole("Waqar");
    System.out.println("name=" + OptionKind.lower(TaglessFinal.greet(OptionKind.MONAD, console)).get());
  }
}
```

Walkthrough Output

```
=== Tagless Final problem: swappable Console ===
name=Waqar
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Tagless Final.

Related Patterns

- Contrast Tagless Final with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → Tagless Final → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.